

PERGUNTAS

Por que um LLM não pode trabalhar diretamente com o texto bruto?

Modelos de linguagem são, na sua essência, grandes calculadoras matriciais. Redes neurais não conseguem interpretar letras, espaços ou strings, elas processam apenas números reais. Além disso, a linguagem humana é cheia de irregularidades, como pontuações coladas em palavras. O texto bruto precisa ser limpo, padronizado e fatiado em unidades tratáveis antes de qualquer operação matemática acontecer.

Qual é a função do vocabulário?

O vocabulário funciona como o dicionário oficial e estático do modelo. Ele mapeia cada pedaço de texto conhecido para um número inteiro único. Como vimos nos experimentos de tokenização, ele também tem uma função de proteção: ao encontrar uma palavra que não está no seu catálogo, ele evita que o sistema quebre mapeando-a para um token de segurança genérico, o `<|unk|>`.

Qual é a diferença entre um token e um Token ID?

O token é o fragmento de texto em si (pode ser uma palavra inteira, uma sílaba ou até um único caractere de pontuação, como uma vírgula). Já o Token ID é o número inteiro (índice) que representa esse token dentro do vocabulário do modelo.

Por que os Token IDs não são utilizados diretamente como representação semântica?

Porque os Token IDs são apenas índices arbitrários. Se a palavra "cachorro" tem o ID 10 e a palavra "gato" tem o ID 1000, alimentar a rede neural diretamente com esses inteiros faria o modelo interpretar erroneamente que um gato "vale cem vezes mais" que um cachorro. Números inteiros isolados não carregam relação de significado, apenas de posição no dicionário.

Qual é a função dos embeddings?

A camada de embedding resolve o problema dos Token IDs transformando cada índice inteiro em um vetor denso (uma lista de números decimais). Em vez de um número único, a palavra passa a ser representada por coordenadas em um espaço multidimensional. Isso permite que a rede neural aproxime os valores matemáticos de palavras que possuem significados parecidos durante o treinamento.

Por que é necessário representar a posição dos tokens?

Diferente das redes neurais recorrentes antigas que liam o texto palavra por palavra, a arquitetura Transformer processa todos os tokens da janela de contexto simultaneamente (em paralelo). Sem a injeção do Positional Embedding, o modelo não saberia diferenciar a frase "O João comeu o bolo" de "O bolo comeu o João", pois os vetores semânticos seriam exatamente os mesmos. Somar o vetor de posição garante que o modelo entenda a ordem temporal da frase.

Qual é a relação entre tamanho do contexto e quantidade de amostras de treinamento?

A relação é inversamente proporcional. Como validamos nos testes de fatiamento do DataLoader, a quantidade de amostras segue a lógica $\text{Amostras} = N - C$, onde N é o total de tokens do documento e C é o tamanho do contexto. Se a janela de contexto for muito grande, o algoritmo consome fatias maiores do texto a cada passo, sobrando menos espaço para deslizar a janela. Portanto, contextos maiores exigem volumes proporcionalmente maiores de texto bruto para gerar amostras suficientes.

Qual é o impacto da dimensão do embedding sobre as estruturas utilizadas pelo modelo?

O aumento da dimensão do embedding causa uma expansão drástica no tamanho do tensor tridimensional gerado. Nos experimentos, observamos que o formato dos dados sai de uma matriz pequena e didática para estruturas massivas. Isso impacta diretamente o hardware, exigindo muito mais memória RAM e VRAM da placa de vídeo para armazenar e multiplicar essas matrizes.

Qual é a função do DataLoader no pipeline?

A função principal é o empacotamento e a paralelização. Em vez de enviar uma sequência de treinamento por vez para a rede neural (o que seria extremamente lento), o DataLoader agrupa várias sequências em um lote (batch). Isso cria a primeira dimensão do tensor final, permitindo que a GPU processe dezenas de exemplos simultaneamente, o que acelera e dá mais estabilidade matemática ao treinamento.

Quais informações produzidas nesta Sprint serão utilizadas pelo mecanismo de atenção da Sprint seguinte?

O produto final desta Sprint é o tensor tridimensional de entrada, cujo formato engloba as dimensões Lote, Contexto e Embedding. Esse tensor, que já carrega a soma do significado das palavras com suas respectivas posições absolutas, é exatamente a estrutura de dados bruta que alimentará as matrizes de *Query*, *Key* e *Value* do mecanismo de atenção (Self-Attention) na próxima etapa do projeto.